

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE CIENCIAS MATEMÁTICAS

ESCUELA PROFESIONAL DE COMPUTACIÓN CIENTÍFICA



MONOGRAFÍA

Capítulo 19

"Teoría del Aprendizaje y Modelos Lineales"

Curso: Inteligencia Artificial

Integrantes: Aguilar Morán, Roussy Jesús
Guerra Velasquez, Jardiny Irazhova
Muñoz Vega, Lucas David
Supo Martínez, Omar Valentín

Docente: López Cordova, Frida Mereyda

Lima – Perú

2026

Índice

1. Introducción	5
2. Formas de Aprendizaje	5
2.1. Tipos de aprendizaje	6
2.1.1. Aprendizaje supervisado	6
2.1.2. Aprendizaje no supervisado	6
2.1.3. Aprendizaje por refuerzo	6
3. Aprendizaje Supervisado	6
3.1. Espacio de hipótesis	6
3.2. Clasificación y Regresión	7
4. Ejemplo: Espera en un Restaurante	7
4.1. Descripción del problema	7
4.2. Naturaleza del problema	8
5. Ejemplo Práctico en Python	8
6. Generalización	9
7. Sesgo y Varianza	10
7.1. Sesgo	10
7.2. Varianza	10
7.3. Compromiso sesgo-varianza	10
8. Selección de Modelos	10
9. Introducción al Aprendizaje Supervisado	11
10. Aprendizaje de Árboles de Decisión	11
11. Representación Lógica e Introspección	12

12. Selección de Atributos y Ganancia de Información	12
13. El Algoritmo de Inducción	13
14. Metodología de Evaluación	14
15. Funciones de Pérdida y Regularización	15
16. La Teoría del Aprendizaje	18
16.1. Ejemplo de aprendizaje PAC: Aprendizaje de listas de decisión	19
17. Regresión Lineal y Clasificación	25
17.1. Regresión lineal univariable	25
17.2. Descenso de gradiente	26
17.3. Regresión lineal multivariable	28
17.4. Clasificadores lineales con un umbral rígido	29
17.5. Clasificación lineal con regresión logística	33
18. Modelos No Paramétricos	36
18.1. Fundamento Teórico	36
18.2. k -Nearest Neighbors (k -NN)	37
18.2.1. Principio de Funcionamiento	37
18.2.2. Métricas de Distancia	37
18.2.3. Elección del Hiperparámetro k	38
18.2.4. Clasificación con k -NN	38
18.2.5. Implementación en Python	38
18.3. Máquinas de Vectores de Soporte (SVM)	39
18.3.1. Principio del Hiperplano Óptimo	39
18.3.2. El Truco del Kernel	40
18.3.3. Implementación en Python	40
18.4. Árboles de Decisión	41
18.4.1. Estructura y Funcionamiento	41
18.4.2. Poda (<i>Pruning</i>)	42

18.5. Naive Bayes	43
18.5.1. Fundamento Probabilístico	43
18.6. Análisis Comparativo de Modelos No Paramétricos	45
19.Ensemble Learning	46
19.1. Concepto y Fundamento Teórico	46
19.2. Random Forest	46
19.2.1. Algoritmo Detallado	46
19.3. AdaBoost (Adaptive Boosting)	48
19.3.1. Fundamento del Boosting	48
19.4. Gradient Boosting	50
19.4.1. Marco General	50
19.5. Stacking (Apilamiento)	52
19.6. Comparación de Métodos Ensemble	54
20.Desarrollo de Sistemas de Machine Learning	55
20.1. El Pipeline Completo de ML	55
20.2. Etapa 1: Definición del Problema y Recolección de Datos	55
20.3. Etapa 2: Análisis Exploratorio y Preprocesamiento	55
20.4. Etapa 3: División de Datos y Validación	57
20.4.1. Estrategias de Validación	57
20.5. Etapa 4: Selección y Entrenamiento del Modelo	58
20.5.1. Estrategia de Selección	58
20.6. Etapa 5: Diagnóstico de Bias y Varianza	60
20.6.1. Curvas de Aprendizaje	60
20.7. Etapa 6: Evaluación Final	61
20.7.1. Métricas de Evaluación	61
20.8. Etapa 7: Despliegue y Monitoreo en Producción	63
20.8.1. Serialización del Modelo	63
20.8.2. Monitoreo del Modelo	64
20.9. Resumen del Pipeline Completo	64

21.Discusión Avanzada	66
21.1. El Balance Bias-Varianza en la Práctica	66
21.2. La Maldición de la Dimensionalidad	66
21.3. Importancia en la Industria	67
22.Conclusiones	68
Referencias	69

1. Introducción

La inteligencia artificial (IA) busca construir sistemas capaces de percibir, razonar, aprender y actuar de manera eficiente en entornos complejos. En este contexto, el aprendizaje automático (Machine Learning) se ha convertido en un componente esencial, ya que permite que los sistemas mejoren su desempeño a partir de la experiencia.

El aprendizaje se basa en la idea de que un agente puede utilizar datos pasados para inferir comportamientos futuros. Este proceso implica pasar de observaciones particulares a reglas generales, lo que se conoce como inducción. A diferencia de la deducción, donde las conclusiones son necesariamente verdaderas si las premisas lo son, la inducción introduce incertidumbre, lo que hace necesario evaluar la calidad de los modelos generados.

2. Formas de Aprendizaje

Los agentes inteligentes están compuestos por múltiples elementos susceptibles de ser mejorados mediante aprendizaje automático. Entre estos se encuentran:

- Mapeo de estados a acciones
- Inferencia de propiedades del entorno
- Modelos de transición del mundo
- Función de utilidad
- Evaluación de acciones
- Definición de metas

Cada uno de estos componentes puede ser aprendido a partir de datos, dependiendo del conocimiento previo del agente y de la disponibilidad de retroalimentación.

2.1. Tipos de aprendizaje

2.1.1. Aprendizaje supervisado

En este enfoque, el agente dispone de ejemplos etiquetados. Cada entrada tiene asociada una salida correcta, lo que permite aprender una función que generaliza a nuevos datos.

2.1.2. Aprendizaje no supervisado

El sistema no recibe etiquetas, por lo que debe descubrir patrones ocultos en los datos. Un ejemplo típico es el agrupamiento de datos similares.

2.1.3. Aprendizaje por refuerzo

El agente interactúa con el entorno y recibe recompensas o castigos. Su objetivo es maximizar la recompensa acumulada a lo largo del tiempo.

3. Aprendizaje Supervisado

Formalmente, el aprendizaje supervisado consiste en encontrar una función $h(x)$ que aproxime una función desconocida $f(x)$ a partir de un conjunto de entrenamiento:

$$h(x) \approx f(x) \tag{1}$$

Donde cada par (x_i, y_i) representa un ejemplo.

3.1. Espacio de hipótesis

El conjunto de funciones posibles que el modelo puede aprender se denomina espacio de hipótesis H . La elección de este espacio es crucial, ya que determina la capacidad del modelo para representar los datos.

Por ejemplo:

- Funciones lineales: simples pero limitadas

- Polinomios: más flexibles pero propensos al sobreajuste
- Modelos complejos: alta capacidad de representación

3.2. Clasificación y Regresión

- Clasificación: salida discreta (ejemplo: verdadero/falso)
- Regresión: salida continua (ejemplo: temperatura)

4. Ejemplo: Espera en un Restaurante

Se estudia un problema donde un agente decide si esperar o no una mesa en un restaurante.

4.1. Descripción del problema

La entrada consiste en un conjunto de atributos discretos:

- Alternativa disponible
- Presencia de bar
- Día (viernes/sábado)
- Nivel de hambre
- Número de clientes
- Precio
- Condición climática
- Reserva
- Tipo de restaurante
- Tiempo estimado de espera

La salida es una variable booleana: esperar o no.

4.2. Naturaleza del problema

Este es un problema de clasificación binaria. A partir de un número reducido de ejemplos, el sistema debe inferir una regla general que funcione para situaciones no vistas.

Esto evidencia uno de los principales retos del aprendizaje automático: la escasez de datos frente a la gran cantidad de combinaciones posibles.

5. Ejemplo Práctico en Python

Se implementa un modelo de árbol de decisión que aprende a partir de datos de entrenamiento.

```
1 from sklearn.tree import DecisionTreeClassifier
2
3 X = [
4     [1,0,0,1,1,3,0,1,0,0],
5     [1,0,0,1,2,1,0,0,1,2],
6     [0,1,0,0,1,1,0,0,2,0],
7     [1,0,1,1,2,1,1,0,1,1],
8     [1,0,1,0,2,3,0,1,0,3],
9     [0,1,0,1,1,2,1,1,2,0],
10    [0,1,0,0,0,1,1,0,2,0]
11 ]
12
13 y = [1,0,1,1,0,1,0]
14
15 modelo = DecisionTreeClassifier(criterion="entropy")
16 modelo.fit(X, y)
17
18 print("Modelo entrenado correctamente.")
19
20 while True:
21     print("\nIngrese los atributos del cliente:")
```

```
22
23     alt = int(input("Alternativa (1=S , 0=No): "))
24     bar = int(input("Bar (1=S , 0=No): "))
25     vie = int(input("Viernes/S bado (1=S , 0=No): "))
26     ham = int(input("Hambriento (1=S , 0=No): "))
27     cli = int(input("Clientes (0=Nada, 1=Algunos, 2=Lleno): "))
28     pre = int(input("Precio (1=$, 2=$$, 3=$$$): "))
29     llu = int(input("Lluvia (1=S , 0=No): "))
30     res = int(input("Reserva (1=S , 0=No): "))
31     tipo = int(input("Tipo (0=Fran,1=Thai,2=Burger,3=Itali): "))
32     est = int(input("Espera (0=0-10,1=10-30,2=30-60,3=>60): "))
33
34     nuevo = [[alt,bar,vie,ham,cli,pre,llu,res,tipo,est]]
35
36     pred = modelo.predict(nuevo)
37
38     if pred[0] == 1:
39         print("Decisi n: Esperar")
40     else:
41         print("Decisi n: No esperar")
42
43     cont = input(" Desea evaluar otro caso? (s/n): ")
44     if cont.lower() != 's':
45         break
```

6. Generalización

Un modelo no debe limitarse a memorizar los datos de entrenamiento. Su verdadero valor radica en su capacidad de generalizar, es decir, de predecir correctamente en datos no vistos.

Para evaluar esto, se utiliza un conjunto de prueba independiente.

7. Sesgo y Varianza

El rendimiento de un modelo depende del equilibrio entre dos factores:

7.1. Sesgo

Se refiere a la capacidad limitada del modelo para capturar patrones complejos. Un alto sesgo conduce al subajuste.

7.2. Varianza

Indica la sensibilidad del modelo a los datos de entrenamiento. Una alta varianza provoca sobreajuste.

7.3. Compromiso sesgo-varianza

Existe una relación inversa entre ambos. Modelos simples tienen alto sesgo y baja varianza, mientras que modelos complejos presentan el comportamiento contrario.

8. Selección de Modelos

La elección del modelo adecuado depende de:

- Naturaleza de los datos
- Cantidad de información disponible
- Complejidad del problema

Un principio fundamental es la navaja de Occam, que sugiere elegir el modelo más simple que explique los datos adecuadamente.

Conclusión

El aprendizaje automático permite a los sistemas adaptarse y mejorar continuamente. La correcta selección del modelo, junto con una adecuada comprensión de conceptos como generalización, sesgo y varianza, es fundamental para el desarrollo de soluciones efectivas.

El ejemplo del restaurante ilustra cómo, incluso con pocos datos, es posible construir modelos predictivos útiles, aunque siempre bajo el desafío de la incertidumbre inherente a la inducción.

9. Introducción al Aprendizaje Supervisado

El aprendizaje automático permite que los agentes computacionales mejoren su rendimiento tras la observación y el análisis de datos provenientes del mundo real.[1] La premisa fundamental reside en la capacidad de una máquina para transformar un conjunto específico de observaciones en reglas generales, un proceso conocido como inducción.[1]

Dentro de este marco, el aprendizaje supervisado se formaliza como la tarea de descubrir una función h (hipótesis) que se aproxime a una función verdadera f desconocida, a partir de un conjunto de entrenamiento compuesto por N pares de entrada-salida $(x_1, y_1), \dots, (x_N, y_N)$. [1, 2]

10. Aprendizaje de Árboles de Decisión

Los árboles de decisión constituyen una de las representaciones más potentes y comprensibles del aprendizaje supervisado. Un árbol es una función que asigna un vector de valores de atributos a una decisión final mediante una secuencia de pruebas que comienzan en la raíz y descienden hasta las hojas.[1, 3]

11. Representación Lógica e Introspección

Un árbol de decisión booleano es equivalente a declaraciones de la lógica proposicional en forma normal disyuntiva (DNF). Cada camino hasta una hoja etiquetada como "verdadero" representa una conjunción de pruebas, y el árbol completo es la disyunción de todos esos caminos.[1]

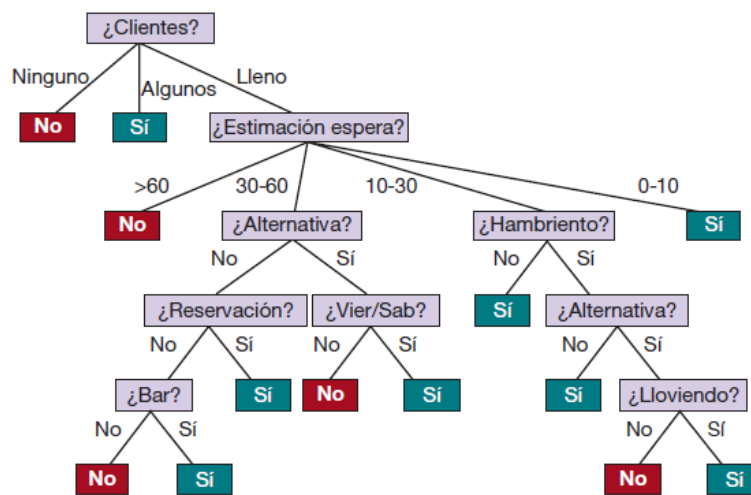


Figura 19.3 Árbol de decisiones para decidir si esperar una mesa.

Figura 1: Árbol de decisión para decidir si esperar por una mesa en un restaurante basado en la introspección humana.[1]

La Figura 1 ilustra cómo un humano codificaría reglas explícitas (ej. "¿Hay hambre?", "¿Qué tipo de comida es?"). Sin embargo, el objetivo del aprendizaje automático es generar estas estructuras automáticamente a partir de datos.

12. Selección de Atributos y Ganancia de Información

El principio subyacente para construir un árbol es seleccionar en cada nodo el atributo que realice la mayor contribución a la clasificación. Esto se cuantifica formalmente mediante la ganancia de información, basada en la entropía de Shannon [1, 4]:

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q)) \quad (2)$$

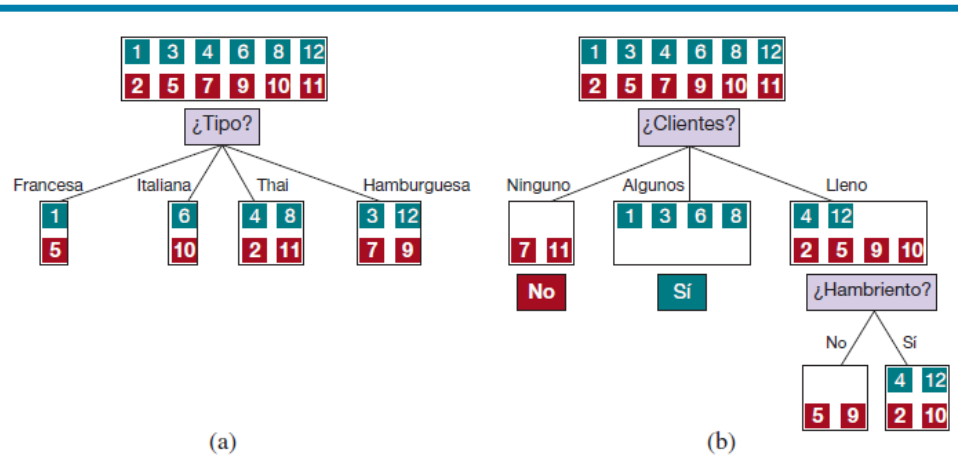


Figura 19.4 Dividiendo los ejemplos mediante pruebas en los atributos. En cada nodo se muestran los ejemplos positivos (cajas claras) y negativos (cajas oscuras) que quedan. (a) Dividir por *Tipo* no nos acerca a distinguir entre ejemplos positivos y negativos. (b) Dividir por *Clientes* hace un buen trabajo al separar ejemplos positivos y negativos. Después de dividir por *Clientes*, *Hambriento* es una segunda prueba bastante buena.

Figura 2: División de ejemplos mediante pruebas de atributos. (a) El atributo 'Tipo' no discrimina bien. (b) El atributo 'Clientes' separa los ejemplos de manera mucho más eficiente.[1]

Como se observa en la Figura 2, el atributo **Clientes** tiene una ganancia de información de 0.541 bits, mientras que **Tipo** tiene 0 bits porque deja los ejemplos mezclados de igual forma en sus ramas.[1]

13. El Algoritmo de Inducción

El algoritmo **LEARN-DECISION-TREE** adopta una estrategia recursiva de "dividir y conquistar".

A diferencia de la Figura 1, el árbol generado por el algoritmo a partir de 12 ejemplos (Figura 4) suele ser más simple, aplicando el principio de la Navaja de Occam: la explicación más sencilla suele ser la mejor.[1]

Selección y Optimización del Modelo

```

function LEARN-DECISION-TREE(examples,attributes,parent_examples) returns un árbol

  if examples está vacío then return PLURALITY-VALUE(parent_examples)
  else if todos los examples tienen la misma clasificación then return la clasificación
  else if attributes está vacío then return PLURALITY-VALUE(examples)
  else
     $A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$ 
    Tree  $\leftarrow$  Un nuevo árbol de decisión con prueba de raíz A
    for each valor v de A do
      exs  $\leftarrow \{e : e \in \text{examples} \text{ and } e.A = v\}$ 
      subtree  $\leftarrow$  LEARN-DECISION-TREE(exs, attributes - A, examples)
      agregue una rama a tree con etiqueta (A = v) y subárbol subtree
    return tree

```

Figura 19.5 Algoritmo de aprendizaje del árbol de decisiones. La función IMPORTANCE se describe en la Sección 19.3.3. La función PLURALITY-VALUE selecciona el valor de salida más común entre un conjunto de ejemplos y deshace los empates de forma aleatoria.

Figura 3: Pseudocódigo del algoritmo de inducción de árboles de decisión.[1]

El objetivo no es ajustarse a los datos pasados, sino seleccionar una hipótesis que mantenga un rendimiento óptimo ante ejemplos futuros.[1]

14. Metodología de Evaluación

Para evitar el sesgo, se dividen los datos en tres partes [1]:

1. **Entrenamiento:** Para construir la hipótesis.[1, 5]
2. **Validación (Dev Set):** Para ajustar hiperparámetros (como la profundidad del árbol).[1]
3. **Prueba:** Para la evaluación final e imparcial.[1]

Cuando los datos son escasos, se emplea la **validación cruzada k-fold**, donde los datos se dividen en k partes y se rota cuál sirve de validación en k rondas.[1, 6]

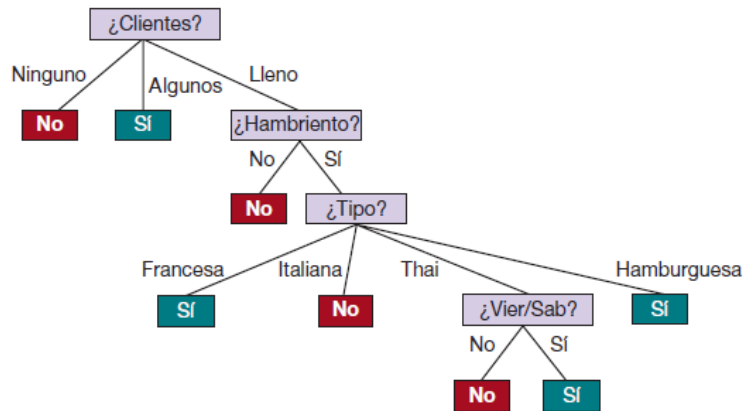


Figura 19.6 El árbol de decisiones inducido a partir del conjunto de entrenamiento de 12 ejemplos.

Figura 4: Árbol de decisión inducido a partir de 12 ejemplos de entrenamiento.[1]

15. Funciones de Pérdida y Regularización

La optimización busca minimizar una función de pérdida $L(y, \hat{y})$. [1, 5] Para evitar el sobreajuste (*overfitting*), se añade un término de regularización:

$$Costo(h) = EmpLoss(h) + \lambda Complexity(h) \quad (3)$$

El parámetro λ controla el balance entre precisión en el entrenamiento y la simplicidad del modelo.[1]

Implementación Práctica en Python

A continuación se presenta el código para replicar el árbol de decisión inducido del restaurante utilizando el dataset de 12 ejemplos del libro.[1]

```

1 import pandas as pd
2 from sklearn . tree import DecisionTreeClassifier , plot_tree
3 import matplotlib . pyplot as plt
4
5 # 1. Definicion del dataset (12 ejemplos de Russell & Norvig )
6 # Mapeo : Yes =1 , No =0. Patrons : None =0 , Some =1 , Full =2.

```

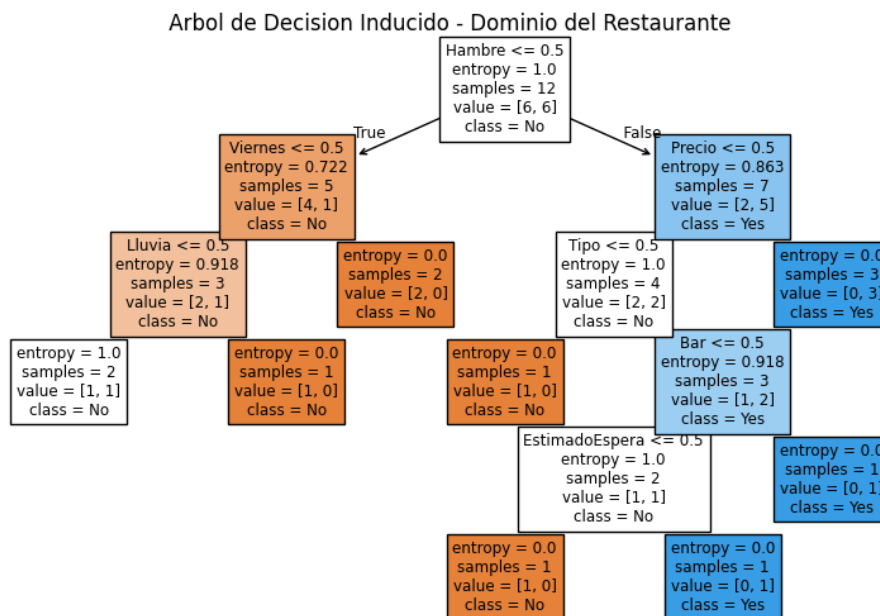


```
7 # Price : $ =0 , $$ =1 , $$$ =2. Type : Fr =0 , Thai =1 , Burg =2 ,
  It =3.
8 # WaitEst : 0 -10=0 , 10 -30=1 , 30 -60=2 , >60=3.
9 data = {
10     'Alt': [1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1],
11     'Bar': [0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0],
12     'Fri': [0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0],
13     'Hun': [1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
14     'Pat': [2, 2, 0, 2, 2, 1, 0, 1, 2, 2, 0, 1],
15     'Price': [2, 0, 0, 0, 2, 1, 0, 0, 2, 1, 0, 0],
16     'Rain': [0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0],
17     'Res': [1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0],
18     'Type': [1, 0, 2, 1, 3, 0, 2, 1, 0, 3, 2, 1],
19     'Est': [0, 0, 0, 1, 1, 2, 0, 0, 1, 2, 0, 0],
20     'Wait': [1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0]
21 }
22
23 df = pd . DataFrame ( data )
24 X = df . drop ( 'Wait' , axis =1)
25 y = df['Wait']
26 # 2. Entrenamiento usando Entropia ( Criterio del libro )
27 clf = DecisionTreeClassifier ( criterion = 'entropy' , random_state
    =42)
28 clf . fit (X , y )
29
30 # Definir nombres de características
31 feature_names_es = [
32     'Alternativa',
33     'Bar',
34     'Viernes',
35     'Hambre',
36     'Patrones',
```

```

37     'Precio',
38     'Lluvia',
39     'Reserva',
40     'Tipo',
41     'EstimadoEspera'
42 ]
43
44 # 3. Visualizacion ( Equivalente a Figura 19.6)
45 plt . figure ( figsize =(10 ,6) )
46 plot_tree ( clf , filled = True , feature_names = feature_names_es ,
47             class_names = ['No', 'Yes'])
48 plt . title ( " Arbol de Decision Inducido - Dominio del Restaurante
49              " )
50 plt . show ()

```



La transición de la lógica humana (Figura 1) a la lógica algorítmica (Figura 4) demuestra que las máquinas pueden encontrar patrones eficientes que nosotros solemos sobrecom-
plicar.[1] El dominio de la selección de modelos y la optimización mediante regularización
garantiza que estos sistemas sean robustos y capaces de generalizar en el mundo real.[1]

16. La Teoría del Aprendizaje

La teoría del aprendizaje estudia cómo garantizar que una hipótesis aprendida h generalice correctamente a ejemplos no vistos, es decir, que sea una buena aproximación de la función objetivo desconocida f . Este problema ha sido analizado tanto desde la filosofía como desde la informática moderna.

En este contexto surgen varias preguntas fundamentales: cuántos ejemplos son necesarios para aprender, qué espacio de hipótesis $\mathcal{H}$ debe utilizarse, qué tan compleja debe ser la hipótesis, si es posible encontrar la mejor solución y cómo evitar el sobreajuste.

La **teoría del aprendizaje computacional** proporciona un marco formal para responder estas preguntas. Su idea central es que una hipótesis que es incorrecta tendrá alta probabilidad de cometer errores al ser evaluada con ejemplos. Por lo tanto, una hipótesis consistente con un número suficiente de ejemplos es poco probable que sea incorrecta.

Este principio se formaliza mediante el concepto de aprendizaje **probablemente aproximadamente correcto (PAC)**. Una hipótesis es PAC si su error es menor o igual que ϵ y esto ocurre con probabilidad al menos $1 - \delta$. Un algoritmo que garantiza este comportamiento se denomina algoritmo de aprendizaje PAC.

El análisis PAC se basa en ciertas suposiciones: los datos se generan a partir de una distribución fija (estacionariedad), la función verdadera f pertenece al espacio de hipótesis $\mathcal{H}$ y es determinista.

El error de una hipótesis h se define como la probabilidad de clasificar incorrectamente un ejemplo nuevo:

$$\text{error}(h) = \sum_{x,y} L_{0/1}(y, h(x))P(x, y)$$

Una hipótesis es aproximadamente correcta si $\text{error}(h) \leq \epsilon$. El conjunto de hipótesis con error mayor que ϵ se denomina $\mathcal{H}_{bad}$.

Para una hipótesis $h_b \in \mathcal{H}_{bad}$, la probabilidad de que sea consistente con N ejemplos es a lo sumo $(1 - \epsilon)^N$. En consecuencia, la probabilidad de que exista al menos una hipótesis

mala consistente en $\mathcal{H}$ está acotada por:

$$P \leq |\mathcal{H}|(1 - \epsilon)^N$$

Para garantizar que esta probabilidad sea menor que δ , el número de ejemplos debe satisfacer:

$$N \geq \frac{1}{\epsilon} \left(\ln \frac{1}{\delta} + \ln |\mathcal{H}| \right)$$

Este valor N se conoce como **complejidad de la muestra**, y representa la cantidad de datos necesarios para asegurar que el aprendizaje sea PAC.

Cuando el espacio de hipótesis es muy grande, como en el caso de todas las funciones booleanas sobre n atributos donde $|\mathcal{H}| = 2^{2^n}$, la complejidad de la muestra crece exponencialmente, lo que implica que se requieren casi todos los ejemplos posibles para aprender correctamente.

Este fenómeno ocurre porque un espacio de hipótesis muy grande puede representar cualquier conjunto de datos de múltiples maneras, lo que impide una generalización efectiva a nuevos ejemplos.

Para abordar este problema se consideran tres enfoques: incorporar conocimiento previo para restringir el espacio de hipótesis, preferir hipótesis más simples y trabajar con subconjuntos restringidos del espacio total que sean más manejables y permitan una mejor generalización.

16.1. Ejemplo de aprendizaje PAC: Aprendizaje de listas de decisión

Las **listas de decisión** constituyen un espacio de hipótesis formado por una secuencia ordenada de pruebas. Cada prueba es una conjunción de literales que, al cumplirse para un ejemplo, determina directamente la salida (*Yes* o *No*). Si la prueba falla, se continúa con la siguiente. A diferencia de los árboles de decisión, las listas de decisión presentan una estructura lineal, aunque sus pruebas pueden ser más complejas.

Una lista de decisión puede representar funciones booleanas, como:

$$WillWait \iff (Patrons = Some) \vee (Patrons = Full \wedge Fri/Sat)$$

Si no se restringe el tamaño de las pruebas, este modelo puede expresar cualquier función booleana. Sin embargo, al limitar las pruebas a un máximo de k literales, se obtiene la clase k -DL, que permite generalizar con menos ejemplos. Esta clase incluye como subconjunto a los árboles de decisión de profundidad máxima k .

Sea k -DL(n) el conjunto de listas de decisión con n atributos booleanos y pruebas de tamaño máximo k . Para analizar su aprendibilidad, se calcula el número de hipótesis posibles. Si $Conj(n, k)$ es el conjunto de conjunciones de hasta k literales, entonces:

$$|Conj(n, k)| = \sum_{i=0}^k \binom{2n}{i} = O(n^k)$$

Dado que cada prueba puede aparecer, no aparecer, o asociarse a un resultado, el número total de listas de decisión está acotado por:

$$|k\text{-DL}(n)| \leq 3^c c! \quad \text{donde } c = |Conj(n, k)|$$

Esto conduce a:

$$|k\text{-DL}(n)| = 2^{O(n^k \log(n^k))}$$

Sustituyendo en la cota PAC, el número de ejemplos necesarios es:

$$N \geq \frac{1}{\epsilon} \left(\ln \frac{1}{\delta} + O(n^k \log(n^k)) \right)$$

Esto muestra que el aprendizaje PAC de k -DL requiere un número de ejemplos polinómico en n , siempre que k sea pequeño.

Para aprender listas de decisión se utiliza el algoritmo DECISION-LIST-LEARNING, que sigue un enfoque ávido. En cada paso selecciona una prueba que cubra un subconjunto no vacío de ejemplos que estén uniformemente clasificados (todos positivos o todos negativos), la añade a la lista y elimina dichos ejemplos. El proceso se repite recursivamente hasta que

no quedan ejemplos.

El algoritmo no especifica un único criterio de selección de pruebas, pero se suele preferir pruebas simples que cubran la mayor cantidad posible de ejemplos homogéneos, con el objetivo de obtener listas compactas. Incluso estrategias simples, como elegir la prueba más pequeña que cubra un subconjunto uniforme, producen buenos resultados en la práctica.

Las curvas de aprendizaje muestran que las listas de decisión logran un buen desempeño con suficiente entrenamiento, aunque en algunos casos los árboles de decisión pueden aprender más rápido o con menor variabilidad.

Ejemplo en Python

A continuación se muestra una implementación simplificada del algoritmo DECISION-LIST-LEARNING:

```
1 # Ejemplo simple de Decision List Learning
2
3 # Datos: [Patrons_Some, Patrons_Full, Fri_Sat] -> WillWait
4 data = [
5     ([1, 0, 0], 1), # Some -> Yes
6     ([1, 0, 1], 1),
7     ([0, 1, 1], 1), # Full AND Fri/Sat -> Yes
8     ([0, 1, 0], 0), # Full pero no Fri/Sat -> No
9     ([0, 0, 0], 0)  # Ninguno -> No
10 ]
11
12 # Generar pruebas simples (conjunciones)
13 def generar_pruebas():
14     pruebas = []
15     # Cada atributo puede ser 1 o 0
16     for i in range(3):
17         pruebas.append([(i, 1)])
18         pruebas.append([(i, 0)])
19
```

```
20     # Conjunciones de 2 atributos
21     pruebas.append([(1,1), (2,1)]) # Full AND Fri/Sat
22     return pruebas
23
24
25 def cumple_prueba(x, prueba):
26     return all(x[i] == val for i, val in prueba)
27
28
29 def decision_list_learning(datos):
30     lista = []
31     datos_restantes = datos.copy()
32
33     while datos_restantes:
34         encontrada = False
35
36         for prueba in generar_pruebas():
37             subset = [(x,y) for x,y in datos_restantes if
38                       cumple_prueba(x, prueba)]
39
40             if not subset:
41                 continue
42
43             etiquetas = [y for _, y in subset]
44
45             # Verificar si todos son iguales
46             if all(y == etiquetas[0] for y in etiquetas):
47                 lista.append((prueba, etiquetas[0]))
48
49             # Eliminar ejemplos cubiertos
50             datos_restantes = [(x,y) for x,y in datos_restantes
51                               if not cumple_prueba(x, prueba)]
```

```
50
51         encontrada = True
52         break
53
54     if not encontrada:
55         print("No se pudo encontrar una prueba consistente")
56         break
57
58     return lista
59
60
61 def predecir(lista, x):
62     for prueba, resultado in lista:
63         if cumple_prueba(x, prueba):
64             return resultado
65     return 0 # default
66
67
68 # Entrenamiento
69 modelo = decision_list_learning(data)
70
71 # Mostrar modelo
72 print("Lista de decision aprendida:")
73 for prueba, res in modelo:
74     print(f"Si {prueba} entonces {res}")
75
76 # Prueba
77 print("\nPredicciones:")
78 test = [[1,0,0], [0,1,1], [0,1,0]]
79 for t in test:
80     print(t, "->", predecir(modelo, t))
```


Explicación del código y resultados

El código implementa una versión simplificada del algoritmo DECISION-LIST-LEARNING para un conjunto de datos booleanos con tres atributos: *Patrons_Some*, *Patrons_Full* y *Fri/Sat*. Cada ejemplo está representado como un vector binario junto con su etiqueta de salida.

En primer lugar, se define el conjunto de datos de entrenamiento, donde cada tupla contiene un ejemplo y su clasificación correspondiente. Luego, la función `generar_pruebas()` construye un conjunto de pruebas candidatas, que incluyen condiciones simples sobre un solo atributo y una conjunción específica de dos atributos.

La función `cumple_prueba(x, prueba)` evalúa si un ejemplo x satisface una prueba dada, verificando que todos los literales de la conjunción se cumplan.

El núcleo del algoritmo se encuentra en `decision_list_learning(datos)`, que construye la lista de decisión de manera iterativa. En cada paso:

- Se recorren las pruebas generadas.
- Para cada prueba, se selecciona el subconjunto de ejemplos que la cumplen.
- Si todos los ejemplos de ese subconjunto tienen la misma etiqueta, la prueba se considera válida.
- Se añade la regla correspondiente a la lista de decisión.
- Se eliminan los ejemplos cubiertos por esa regla.

Este proceso continúa hasta que no quedan ejemplos por clasificar o no se puede encontrar una prueba consistente.

La función `predecir(lista, x)` evalúa un nuevo ejemplo recorriendo la lista de decisión en orden, devolviendo el resultado de la primera prueba que se cumpla.

Al ejecutar el código, el algoritmo aprende una lista de decisión que captura las regularidades del conjunto de entrenamiento. Un resultado típico es una lista de reglas del tipo:

Si $[(0, 1)]$ entonces 1

Si [(1, 1), (2, 1)] entonces 1

Si [(1, 1)] entonces 0

Estas reglas pueden interpretarse como:

- Si *Patrons = Some*, entonces *WillWait = Yes*.
- Si *Patrons = Full* y *Fri/Sat = True*, entonces *WillWait = Yes*.
- Si *Patrons = Full*, entonces *WillWait = No*.

Posteriormente, el modelo se evalúa sobre nuevos ejemplos de prueba. Por ejemplo:

[1,0,0] -> 1

[0,1,1] -> 1

[0,1,0] -> 0

Estos resultados muestran que el modelo clasifica correctamente los ejemplos de acuerdo con las reglas aprendidas, reproduciendo el comportamiento esperado de la función objetivo.

17. Regresión Lineal y Clasificación

Se introduce un nuevo espacio de hipótesis basado en **funciones lineales**, utilizadas para modelar relaciones entre variables continuas. El caso más simple es la regresión lineal univariable, que consiste en ajustar una recta a un conjunto de datos.

17.1. Regresión lineal univariable

Una función lineal univariable tiene la forma:

$$h_{\mathbf{w}}(x) = w_1x + w_0$$

donde $\mathbf{w} = \langle w_0, w_1 \rangle$ son los pesos del modelo.

El objetivo de la regresión lineal es encontrar los valores de $\mathbf{w}$ que mejor ajusten los datos de entrenamiento. En la Figura 19.13(a) se muestra un conjunto de datos donde se relaciona el tamaño de una casa con su precio, junto con la recta ajustada.

Para medir qué tan bien se ajusta la recta a los datos, se utiliza la **función de pérdida cuadrática**:

$$Loss(h_{\mathbf{w}}) = \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2$$

El problema consiste en minimizar esta función de pérdida:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} Loss(h_{\mathbf{w}})$$

La solución se obtiene derivando respecto a los pesos e igualando a cero, lo que produce una solución analítica única:

$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2}, \quad w_0 = \frac{\sum y_j - w_1 \sum x_j}{N}$$

En el ejemplo de la Figura 19.13(a), se obtiene la recta:

$$y = 0,232x + 246$$

Para comprender mejor este proceso, se introduce el concepto de **espacio de pesos**, que representa todas las posibles combinaciones de w_0 y w_1 . En la Figura 19.13(b) se muestra la función de pérdida en este espacio.

La gráfica de la pérdida tiene forma de superficie **convexa**, lo que implica que:

- Existe un único mínimo global.
- No hay mínimos locales.
- Es posible encontrar la mejor solución de manera directa.

Este resultado garantiza que el problema de regresión lineal con pérdida cuadrática tiene una solución óptima única y que el ajuste de la recta puede realizarse de manera eficiente.

17.2. Descenso de gradiente

El modelo lineal univariable tiene la agradable propiedad de que es fácil encontrar una solución óptima donde las derivadas parciales son cero. Pero ese no será siempre el caso, por lo

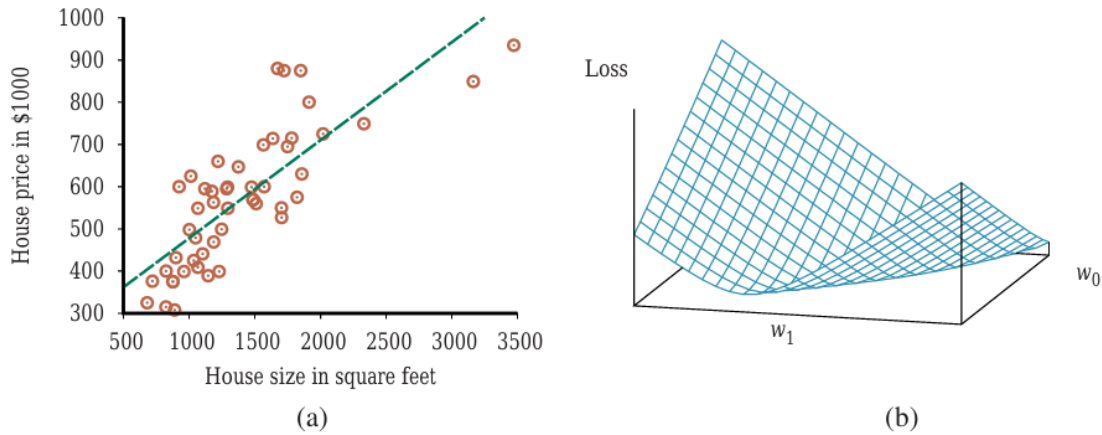


Figura 5: (a) Datos de precio vs. tamaño de casas junto con la función lineal ajustada. (b) Función de pérdida $Loss(y_j - w_1x_j + w_0)^2$ para diferentes valores de w_0 y w_1 . La función de pérdida es convexa y tiene un único mínimo global.

que introducimos aquí un método para minimizar la pérdida que no depende de resolver para encontrar los ceros de las derivadas, y que puede aplicarse a cualquier función de pérdida, sin importar cuán compleja sea.

Como se discutió en la Sección 4.2 (página 119), podemos buscar a través de un espacio de pesos continuo modificando incrementalmente los parámetros. Allí llamamos al algoritmo **búsqueda de ascenso de colinas** (*hill climbing*), pero aquí estamos minimizando la pérdida, no maximizando la ganancia, por lo que usaremos el término **descenso de gradiente**. Elegimos cualquier punto de partida en el espacio de pesos —aquí, un punto en el plano (w_0, w_1) — y luego calculamos una estimación del gradiente y nos movemos una pequeña cantidad en la dirección de descenso más pronunciado, repitiendo hasta que converjamos en un punto en el espacio de pesos con una pérdida mínima (local).

17.3. Regresión lineal multivariable

La regresión lineal multivariable extiende el modelo lineal al caso donde cada entrada es un vector $\mathbf{x} \in R^n$. La hipótesis se define como:

$$h_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_i w_i x_i = \mathbf{w}^\top \mathbf{x}$$

donde se puede incluir el término w_0 como parte del vector de pesos añadiendo un atributo constante.

El objetivo es encontrar el vector de pesos $\mathbf{w}^*$ que minimiza la pérdida cuadrática:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$$

Este problema puede resolverse mediante descenso de gradiente o analíticamente usando la **ecuación normal**:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

En espacios de alta dimensión puede aparecer **sobreajuste**, por lo que se introduce la **regularización**, que penaliza la complejidad del modelo:

$$Cost(h) = EmpLoss(h) + \lambda Complexity(h)$$

Para modelos lineales, la complejidad se define en función de los pesos:

$$Complexity(\mathbf{w}) = \sum_i |w_i|^q$$

donde destacan dos casos:

- L_1 : suma de valores absolutos
- L_2 : suma de cuadrados

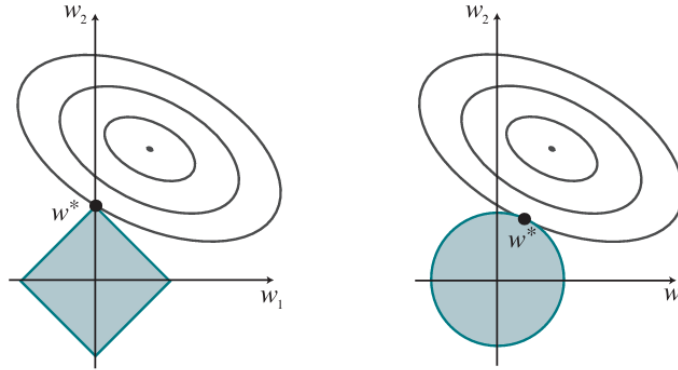


Figure 19.14 Why L_1 regularization tends to produce a sparse model. Left: With L_1 regularization (box), the minimal achievable loss (concentric contours) often occurs on an axis, meaning a weight of zero. Right: With L_2 regularization (circle), the minimal loss is likely to occur anywhere on the circle, giving no preference to zero weights.

Figura 6: Comparación entre regularización L_1 y L_2 . Izquierda: L_1 (forma de diamante) favorece soluciones en los ejes, produciendo pesos cero y modelos dispersos. Derecha: L_2 (forma circular) no favorece pesos cero, distribuyendo los valores entre todas las variables.

La regularización L_1 tiende a generar **modelos dispersos**, es decir, con muchos pesos iguales a cero, lo que equivale a eliminar atributos irrelevantes. Esto ocurre porque la región de restricción tiene forma de diamante, cuyas esquinas coinciden con los ejes.

Por el contrario, la regularización L_2 utiliza una región circular, por lo que no favorece soluciones en los ejes y distribuye los pesos entre todas las variables. En consecuencia, L_1 facilita la selección de características, mientras que L_2 produce modelos más suaves y estables.

17.4. Clasificadores lineales con un umbral rígido

Las funciones lineales también pueden utilizarse para problemas de **clasificación**. En este caso, el objetivo es separar datos en dos clases mediante una **frontera de decisión**. En la Figura 7, se muestran dos clases de datos (por ejemplo, terremotos y explosiones), representadas en función de dos atributos x_1 y x_2 .

Una frontera de decisión lineal tiene la forma de una recta:

$$w_0 + w_1x_1 + w_2x_2 = 0$$

y define un **separador lineal**. Los datos que pueden separarse de esta forma se denominan **linealmente separables**.

El modelo puede expresarse como un producto punto:

$$\mathbf{w} \cdot \mathbf{x} = w_0x_0 + w_1x_1 + w_2x_2$$

donde $x_0 = 1$. La clasificación se obtiene aplicando una **función de umbral**:

$$h_{\mathbf{w}}(\mathbf{x}) = \begin{cases} 1 & \text{si } \mathbf{w} \cdot \mathbf{x} \geq 0 \\ 0 & \text{en otro caso} \end{cases}$$

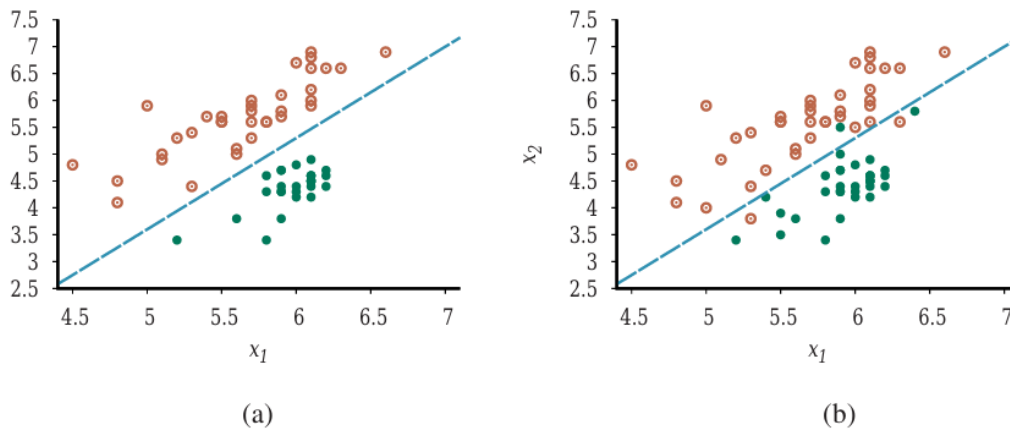


Figura 7: (a) Datos linealmente separables con una frontera de decisión lineal. (b) Datos no linealmente separables, donde una única recta no puede separar perfectamente las clases.

A diferencia de la regresión, la función de umbral no es diferenciable, por lo que no es posible aplicar directamente métodos basados en gradiente. En su lugar, se utiliza la **regla de aprendizaje del perceptrón** para ajustar los pesos:

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x}))x_i$$

Esta regla actualiza los pesos de la siguiente manera:

- Si la predicción es correcta, no se modifican los pesos.
- Si el ejemplo debería ser positivo pero se clasifica como negativo, los pesos aumentan.

- Si el ejemplo debería ser negativo pero se clasifica como positivo, los pesos disminuyen.

El algoritmo converge a una solución correcta siempre que los datos sean linealmente separables. En caso contrario, no existe un separador lineal perfecto, como se observa en la Figura 7(b).

Ejemplo en Python

A continuación se muestra una implementación de un clasificador lineal utilizando la regla de aprendizaje del perceptrón:

```
1 # Ejemplo de clasificador lineal con perceptrón
2
3 import numpy as np
4
5 # Datos de entrenamiento (x1, x2) -> clase (0 o 1)
6 X = np.array([
7     [5.0, 6.0],
8     [6.0, 6.5],
9     [5.5, 5.8],
10    [6.2, 6.2],
11    [5.8, 6.1], # clase 0
12
13    [5.5, 4.0],
14    [6.0, 4.5],
15    [5.8, 4.2],
16    [6.2, 4.8],
17    [5.7, 4.3] # clase 1
18 ])
19
20 y = np.array([0,0,0,0,0, 1,1,1,1,1])
21
22 # Agregar bias (x0 = 1)
23 X = np.hstack((np.ones((X.shape[0], 1)), X))
```



```
24
25 # Inicializar pesos
26 w = np.zeros(X.shape[1])
27
28 def predict(x):
29     return 1 if np.dot(w, x) >= 0 else 0
30
31 # Entrenamiento
32 alpha = 0.1
33 epochs = 20
34
35 for epoch in range(epochs):
36     for i in range(len(X)):
37         y_pred = predict(X[i])
38         error = y[i] - y_pred
39         w = w + alpha * error * X[i]
40
41 print("Pesos aprendidos:", w)
42
43 # Pruebas
44 test = np.array([
45     [1, 5.5, 6.0],
46     [1, 6.0, 4.2],
47     [1, 5.8, 5.0]
48 ])
49
50 for t in test:
51     print(t[1:], "->", predict(t))
```

Explicación del código El código implementa un clasificador lineal basado en el algoritmo del perceptrón. Primero se define un conjunto de datos con dos características (x_1, x_2) y sus

respectivas clases (0 o 1). Luego se añade un término constante $x_0 = 1$ para incluir el sesgo en el modelo.

El vector de pesos $\mathbf{w}$ se inicializa en cero y se ajusta iterativamente utilizando la regla de aprendizaje del perceptrón:

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x}))x_i$$

Durante el entrenamiento, el algoritmo recorre los datos varias veces (épocas). Si la predicción es incorrecta, los pesos se ajustan para corregir el error; si es correcta, no se modifican.

Resultado generado Como resultado, el algoritmo aprende un vector de pesos $\mathbf{w}$ que define una frontera de decisión lineal de la forma:

$$w_0 + w_1x_1 + w_2x_2 = 0$$

Esta frontera separa las dos clases del conjunto de entrenamiento. Finalmente, el modelo se utiliza para predecir la clase de nuevos ejemplos. Si los datos son linealmente separables, el perceptrón converge a una solución que clasifica correctamente todos los ejemplos.

17.5. Clasificación lineal con regresión logística

El uso de una función de umbral rígido en clasificadores lineales presenta limitaciones importantes, ya que produce una hipótesis no diferenciable y discontinua. Esto dificulta el aprendizaje mediante métodos basados en gradiente y genera predicciones completamente deterministas (0 o 1), incluso para ejemplos cercanos a la frontera de decisión.

Para solucionar estos problemas, se introduce una versión suavizada del umbral mediante una función continua y diferenciable: la **función logística** o sigmoide:

$$\text{Logistic}(z) = \frac{1}{1 + e^{-z}}$$

Sustituyendo la función de umbral por la función logística, la hipótesis se define como:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Logistic}(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{x}}}$$

Este modelo produce valores en el intervalo $(0, 1)$, lo que permite interpretar la salida como una probabilidad. De esta manera, la regresión logística no solo clasifica, sino que también proporciona una medida de confianza en la predicción.

Además, al ser una función diferenciable, permite el uso de métodos de optimización como el descenso de gradiente para ajustar los pesos de forma más estable y eficiente.

Ejemplo en Python

A continuación se muestra una implementación de un clasificador lineal utilizando regresión logística:

```
1 # Ejemplo de regresión logística
2
3 import numpy as np
4
5 X = np.array([
6     [5.0, 6.0],
7     [6.0, 6.5],
8     [5.5, 5.8],
9     [6.2, 6.2],
10    [5.8, 6.1],
11
12    [5.5, 4.0],
13    [6.0, 4.5],
14    [5.8, 4.2],
15    [6.2, 4.8],
16    [5.7, 4.3]
17 ])
18
19 y = np.array([0,0,0,0,0, 1,1,1,1,1])
20
21 X = np.hstack((np.ones((X.shape[0], 1)), X))
22
```

```
23 w = np.zeros(X.shape[1])
24
25 def sigmoid(z):
26     return 1 / (1 + np.exp(-z))
27
28 def predict_proba(x):
29     return sigmoid(np.dot(w, x))
30
31 def predict(x):
32     return 1 if predict_proba(x) >= 0.5 else 0
33
34 alpha = 0.1
35 epochs = 1000
36
37 for epoch in range(epochs):
38     for i in range(len(X)):
39         y_pred = predict_proba(X[i])
40         error = y[i] - y_pred
41         w = w + alpha * error * X[i]
42
43 print("Pesos aprendidos:", w)
44
45 test = np.array([
46     [1, 5.5, 6.0],
47     [1, 6.0, 4.2],
48     [1, 5.8, 5.0]
49 ])
50
51 for t in test:
52     print(t[1:], "-> prob:", predict_proba(t), "clase:", predict(t))
```

Explicación del código El código implementa un clasificador basado en regresión logística. A diferencia del perceptrón, utiliza la función sigmoide para producir valores entre 0 y 1, los cuales pueden interpretarse como probabilidades.

Durante el entrenamiento, los pesos se ajustan mediante descenso de gradiente, reduciendo el error entre la salida esperada y la predicción probabilística.

Resultado generado El modelo aprende un vector de pesos que define una frontera de decisión lineal. Para cada nuevo ejemplo, primero calcula una probabilidad y luego asigna una clase utilizando un umbral de 0.5. Esto permite no solo clasificar, sino también medir la confianza de la predicción.

18. Modelos No Paramétricos

18.1. Fundamento Teórico

Los modelos **no paramétricos** representan una familia de técnicas de aprendizaje automático que no asumen una forma funcional fija para la relación entre variables. A diferencia de los modelos paramétricos (como la regresión lineal, que asume una relación lineal), los modelos no paramétricos permiten que el propio modelo descubra la estructura subyacente de los datos.

Formalmente, sea $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ un conjunto de entrenamiento. Un modelo no paramétrico busca aproximar una función desconocida $f : \mathcal{X} \rightarrow \mathcal{Y}$ sin restringirla a una familia paramétrica específica:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \sum_{i=1}^n L(y_i, f(x_i)) + \lambda \Omega(f)$$

donde L es una función de pérdida, $\Omega(f)$ es un término de regularización y λ controla el balance entre ajuste y complejidad.

Ventajas principales:

- Flexibilidad para capturar relaciones no lineales complejas.

- No requieren suposiciones sobre la distribución de los datos.
- Se adaptan automáticamente a la complejidad inherente del problema.

Desventajas principales:

- Mayor costo computacional en predicción (especialmente k -NN).
- Riesgo de *overfitting* si no se regulariza adecuadamente.
- La maldición de la dimensionalidad afecta severamente su rendimiento.

18.2. k -Nearest Neighbors (k -NN)

18.2.1. Principio de Funcionamiento

El algoritmo k -NN es un método basado en instancias (*instance-based learning*) que clasifica o predice un nuevo punto x_q basándose en los k ejemplos más cercanos en el espacio de características. Es uno de los algoritmos más intuitivos: la predicción se realiza en función de la “vecindad” del punto de consulta.

El proceso completo es:

1. **Calcular distancias:** se mide la distancia entre x_q y todos los puntos de entrenamiento.
2. **Seleccionar vecinos:** se identifican los k puntos más cercanos $\mathcal{N}_k(x_q)$.
3. **Inferir la predicción:** por votación mayoritaria (clasificación) o promedio (regresión).

18.2.2. Métricas de Distancia

La elección de la métrica de distancia es crítica para el rendimiento del algoritmo:

Distancia Euclidiana (la más común):

$$d_E(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}$$

Distancia Manhattan:

$$d_M(x_i, x_j) = \sum_{k=1}^p |x_{ik} - x_{jk}|$$

Distancia de Minkowski (generalización):

$$d_p(x_i, x_j) = \left(\sum_{k=1}^p |x_{ik} - x_{jk}|^p \right)^{1/p}$$

Distancia coseno (útil para texto):

$$d_{\cos}(x_i, x_j) = 1 - \frac{x_i \cdot x_j}{\|x_i\| \cdot \|x_j\|}$$

18.2.3. Elección del Hiperparámetro k

El valor de k controla el balance entre *bias* y *varianza*:

- k pequeño (ej. $k = 1$): baja *bias*, alta *varianza* → riesgo de *overfitting*.
- k grande: alta *bias*, baja *varianza* → riesgo de *underfitting*.
- Se recomienda usar validación cruzada para seleccionar k óptimo.
- Típicamente se prueba con valores impares para evitar empates en clasificación binaria.

18.2.4. Clasificación con k -NN

$$\hat{y} = \arg \max_c \sum_{(x_i, y_i) \in \mathcal{N}_k(x_q)} 1[y_i = c]$$

18.2.5. Implementación en Python

Listing 1: Implementación completa de k -NN con scikit-learn

```

1 from sklearn.neighbors import KNeighborsClassifier
2 from sklearn.preprocessing import StandardScaler

```

```

3 from sklearn.model_selection import cross_val_score, GridSearchCV
4 import numpy as np
5
6 # Preprocesamiento: normalizaci n esencial para k-NN
7 scaler = StandardScaler()
8 X_train_scaled = scaler.fit_transform(X_train)
9 X_test_scaled = scaler.transform(X_test)
10
11 # B squeda del k ptimo mediante validaci n cruzada
12 param_grid = {'n_neighbors': range(1, 21),
13               'metric': ['euclidean', 'manhattan']}
14 grid_search = GridSearchCV(KNeighborsClassifier(), param_grid,
15                             cv=5, scoring='accuracy')
16 grid_search.fit(X_train_scaled, y_train)
17
18 print(f"Mejor k: {grid_search.best_params_}")
19 print(f"Accuracy CV: {grid_search.best_score_:.4f}")
20
21 # Modelo final con mejor k
22 model = grid_search.best_estimator_
23 y_pred = model.predict(X_test_scaled)

```

18.3. Máquinas de Vectores de Soporte (SVM)

18.3.1. Principio del Hiperplano Óptimo

Las SVM buscan encontrar el hiperplano que maximiza el **margen** entre las clases. El margen es la distancia entre el hiperplano de decisión y los puntos de entrenamiento más cercanos de cada clase (llamados **vectores de soporte**).

El problema de optimización primal es:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{sujeto a} \quad y_i(w \cdot x_i + b) \geq 1, \quad \forall i$$

Para datos no linealmente separables (SVM de margen suave), se introducen variables de holgura $\xi_i \geq 0$:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{sueto a} \quad y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

donde C es el parámetro de regularización que controla el trade-off entre maximizar el margen y minimizar los errores de clasificación.

18.3.2. El Truco del Kernel

El poder de las SVM radica en el **truco del kernel**: mediante una función $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$, se puede mapear implícitamente los datos a un espacio de alta dimensión sin calcular explícitamente la transformación ϕ .

Kernels más utilizados:

Cuadro 1: Kernels más comunes en SVM

Kernel	Fórmula	Uso Típico
Lineal	$K(x_i, x_j) = x_i \cdot x_j$	Datos linealmente separables
Polinomial	$K(x_i, x_j) = (\gamma x_i \cdot x_j + r)^d$	Relaciones polinómicas
RBF (Gaussiano)	$K(x_i, x_j) = \exp(-\gamma \ x_i - x_j\ ^2)$	Uso general, muy popular
Sigmoide	$K(x_i, x_j) = \tanh(\gamma x_i \cdot x_j + r)$	Redes neuronales

18.3.3. Implementación en Python

Listing 2: SVM con búsqueda de hiperparámetros

```

1 from sklearn.svm import SVC
2 from sklearn.model_selection import GridSearchCV
3 from sklearn.preprocessing import StandardScaler
4
5 # Las SVM son sensibles a la escala
6 scaler = StandardScaler()
7 X_train_scaled = scaler.fit_transform(X_train)

```

```
8
9 # Búsqueda de hiperparámetros ptimos
10 param_grid = {
11     'C': [0.1, 1, 10, 100],
12     'kernel': ['rbf', 'poly', 'linear'],
13     'gamma': ['scale', 'auto', 0.001, 0.01]
14 }
15
16 svm_grid = GridSearchCV(SVC(probability=True), param_grid,
17                          cv=5, scoring='f1_weighted', n_jobs=-1)
18 svm_grid.fit(X_train_scaled, y_train)
19
20 print(f"Mejores parámetros: {svm_grid.best_params_}")
21 best_svm = svm_grid.best_estimator_
```

18.4. Árboles de Decisión

18.4.1. Estructura y Funcionamiento

Los árboles de decisión son modelos jerárquicos que aprenden reglas de decisión inferidas de los datos. Cada nodo interno representa una prueba sobre un atributo, cada rama representa el resultado de la prueba, y cada nodo hoja representa una clase o valor.

El algoritmo de aprendizaje busca la división (*split*) que maximiza la **pureza** de los subconjuntos resultantes. Las métricas más usadas son:

Índice de Gini (usado por CART):

$$\text{Gini}(t) = 1 - \sum_{k=1}^K p_k^2$$

donde p_k es la proporción de ejemplos de la clase k en el nodo t .

Entropía (usada por ID3/C4.5):

$$H(t) = - \sum_{k=1}^K p_k \log_2 p_k$$

Ganancia de información:

$$IG(t, a) = H(t) - \sum_{v \in \text{valores}(a)} \frac{|t_v|}{|t|} H(t_v)$$

18.4.2. Poda (*Pruning*)

Para evitar el *overfitting*, los árboles de decisión se pueden podar:

- **Pre-poda:** detener el crecimiento del árbol antes de que sea perfecto (límite de profundidad, mínimo de muestras por nodo).
- **Post-poda:** crecer el árbol completamente y luego eliminar ramas que no mejoran el rendimiento en validación.

Listing 3: Árbol de decisión con control de sobreajuste

```

1 from sklearn.tree import DecisionTreeClassifier, plot_tree
2 from sklearn.model_selection import cross_val_score
3 import matplotlib.pyplot as plt
4
5 # Controlar profundidad para evitar overfitting
6 model = DecisionTreeClassifier(
7     criterion='gini',
8     max_depth=5,
9     min_samples_split=10,
10    min_samples_leaf=5,
11    random_state=42
12 )
13 model.fit(X_train, y_train)
14

```

```

15 # Evaluar con validaci n cruzada
16 cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
17 print(f"CV Accuracy: {cv_scores.mean():.4f} +/- {cv_scores.std():.4f}
    ")
18
19 # Visualizar el rbol
20 plt.figure(figsize=(15, 8))
21 plot_tree(model, filled=True, feature_names=feature_names,
22           class_names=class_names, rounded=True)
23 plt.title(" rbol de Decisi n")
24 plt.savefig("arbol.png", dpi=150, bbox_inches='tight')

```

18.5. Naive Bayes

18.5.1. Fundamento Probabilístico

Naive Bayes es un clasificador probabilístico basado en el **Teorema de Bayes**, que asume independencia condicional entre las características dado la clase:

$$P(y \mid x_1, x_2, \dots, x_p) = \frac{P(y) \prod_{i=1}^p P(x_i \mid y)}{P(x_1, x_2, \dots, x_p)}$$

La predicción se realiza con la regla MAP (*Maximum a Posteriori*):

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^p P(x_i \mid y)$$

Variantes principales:

- **Gaussian NB:** $P(x_i \mid y)$ sigue una distribución normal $\rightarrow$ para características continuas.
- **Multinomial NB:** para conteos de palabras $\rightarrow$ texto y NLP.
- **Bernoulli NB:** para características binarias.
- **Complement NB:** variante robusta para clases desbalanceadas.

Listing 4: Naive Bayes para clasificación de texto

```
1 from sklearn.naive_bayes import GaussianNB, MultinomialNB
2 from sklearn.pipeline import Pipeline
3 from sklearn.feature_extraction.text import TfidfVectorizer
4
5 # Para datos num ricos continuos
6 gnb = GaussianNB(var_smoothing=1e-9)
7 gnb.fit(X_train, y_train)
8 print(f"Accuracy GNB: {gnb.score(X_test, y_test):.4f}")
9
10 # Pipeline para clasificaci n de texto
11 text_clf = Pipeline([
12     ('tfidf', TfidfVectorizer(max_features=5000)),
13     ('clf', MultinomialNB(alpha=1.0)) # alpha: suavizado de Laplace
14 ])
15 text_clf.fit(X_text_train, y_train)
```

18.6. Análisis Comparativo de Modelos No Paramétricos

Cuadro 2: Comparación de modelos no paramétricos

Modelo	Fortalezas	Debilidades	Mejor caso de uso
<i>k</i> -NN	Simple, sin entrenamiento, interpretable	Lento en predicción, sensible a ruido y dimensionalidad	Datasets pequeños, prototipos
SVM	Alto rendimiento, funciona bien en alta dimensionalidad, marginal máximo	Difícil de escalar, sensible a hiperparámetros	Clasificación binaria, texto
Árbol de Decisión	Interpretable, maneja valores mixtos, no requiere normalización	Sobreajuste, inestable	Problemas donde la interpretabilidad es clave
Naive Bayes	Muy rápido, funciona con pocos datos, robusto	Asunción de independencia irrealista	Clasificación de texto, spam

19. Ensemble Learning

19.1. Concepto y Fundamento Teórico

El **Ensemble Learning** (aprendizaje en conjunto) es una técnica que combina múltiples modelos base (*weak learners*) para producir un modelo más robusto y preciso (*strong learner*). La intuición es análoga a la toma de decisiones colectiva: un grupo de expertos con distintas perspectivas tiende a tomar mejores decisiones que cualquier individuo por separado.

Matemáticamente, dado un conjunto de modelos $h_1, h_2, \dots, h_T$, el ensemble produce:

$$H(x) = \text{Combinación}(h_1(x), h_2(x), \dots, h_T(x))$$

El fundamento teórico se basa en la descomposición **Bias-Varianza-Ruido**:

$$\text{Error}(f) = \text{Bias}^2 + \text{Varianza} + \text{Ruido irreducible}$$

- **Bagging** reduce la varianza al promediar múltiples modelos entrenados en subconjuntos bootstrap.
- **Boosting** reduce el bias entrenando iterativamente modelos que corrigen los errores del anterior.
- **Stacking** puede reducir ambos mediante meta-aprendizaje.

19.2. Random Forest

19.2.1. Algoritmo Detallado

Random Forest extiende el Bagging de árboles de decisión con una modificación crucial: en cada división del árbol, solo se considera un subconjunto aleatorio de m características (donde típicamente $m = \sqrt{p}$ para clasificación y $m = p/3$ para regresión, siendo p el total de características).

Algoritmo:

1. Para $t = 1, 2, \dots, T$:
 - a) Obtener muestra bootstrap D_t de tamaño n con reemplazo del conjunto de entrenamiento.
 - b) Entrenar árbol h_t en D_t : en cada nodo, seleccionar la mejor división entre m características aleatorias.
2. Para clasificación: $H(x) = \text{voto mayoritario}\{h_t(x)\}_{t=1}^T$
3. Para regresión: $H(x) = \frac{1}{T} \sum_{t=1}^T h_t(x)$

Ventaja clave - Importancia de Variables: Random Forest permite calcular la importancia de cada característica midiendo cuánto disminuye la impureza del nodo cuando se usa esa característica:

$$\text{Importance}(x_j) = \frac{1}{T} \sum_{t=1}^T \sum_{\text{nodos } n \text{ en } h_t} 1[\text{split en } x_j] \cdot \Delta \text{Impureza}(n)$$

Listing 5: Random Forest con análisis de importancia de variables

```

1 from sklearn.ensemble import RandomForestClassifier
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 # Modelo Random Forest
7 rf = RandomForestClassifier(
8     n_estimators=200,
9     max_depth=None,          # rboles completos (la aleatoriedad evita
                             overfitting)
10    min_samples_split=2,
11    max_features='sqrt',     # sqrt(p) características por split
12    bootstrap=True,
13    oob_score=True,          # Error Out-of-Bag (estimación gratuita
                             del error)

```



```
14     n_jobs=-1,
15     random_state=42
16 )
17 rf.fit(X_train, y_train)
18
19 # Error OOB: estimaci n de generalizaci n sin necesidad de
    validaci n
20 print(f"OOB Score: {rf.oob_score_:.4f}")
21 print(f"Test Accuracy: {rf.score(X_test, y_test):.4f}")
22
23 # Importancia de variables
24 feat_importance = pd.Series(rf.feature_importances_,
25                             index=feature_names).sort_values(
26                             ascending=False)
27 print("\nTop 10 caracter sticas m s importantes:")
28
29 # Visualizaci n
30 plt.figure(figsize=(10, 6))
31 feat_importance.head(15).plot(kind='barh', color='steelblue')
32 plt.title("Importancia de Variables - Random Forest")
33 plt.xlabel("Importancia (Mean Decrease in Impurity)")
34 plt.tight_layout()
35 plt.savefig("importancia_variables.png", dpi=150)
```

19.3. AdaBoost (Adaptive Boosting)

19.3.1. Fundamento del Boosting

AdaBoost es el algoritmo de Boosting clásico propuesto por Freund y Schapire (1997). La idea central es entrenar iterativamente clasificadores débiles, prestando mayor atención a los ejemplos mal clasificados en iteraciones anteriores.

Algoritmo AdaBoost:

1. Inicializar pesos uniformes: $w_i^{(1)} = 1/n$ para todo i .
2. Para $t = 1, 2, \dots, T$:
 - a) Entrenar clasificador h_t con pesos $\{w_i^{(t)}\}$.
 - b) Calcular error ponderado: $\varepsilon_t = \sum_{i:h_t(x_i) \neq y_i} w_i^{(t)}$
 - c) Calcular peso del clasificador: $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\varepsilon_t}{\varepsilon_t} \right)$
 - d) Actualizar pesos: $w_i^{(t+1)} \propto w_i^{(t)} \cdot e^{-\alpha_t y_i h_t(x_i)}$
3. Predicción final: $H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$

Los clasificadores con menor error reciben mayor peso α_t en la decisión final.

Listing 6: AdaBoost con análisis de convergencia

```

1 from sklearn.ensemble import AdaBoostClassifier
2 from sklearn.tree import DecisionTreeClassifier
3 import numpy as np
4
5 # AdaBoost con stumps de decisi n (clasificadores d biles por
   defecto)
6 adaboost = AdaBoostClassifier(
7     estimator=DecisionTreeClassifier(max_depth=1), # Stump
8     n_estimators=200,
9     learning_rate=0.1, # Shrinkage: reduce la contribuci n de cada
   clasificador
10    algorithm='SAMME',
11    random_state=42
12 )
13 adaboost.fit(X_train, y_train)
14
15 # An lisis de convergencia: error en funci n del n mero de
   estimadores

```

```
16 staged_scores = list(adaboost.staged_score(X_test, y_test))
17 print(f"Accuracy con 100 estimadores: {staged_scores[99]:.4f}")
18 print(f"Accuracy con 200 estimadores: {staged_scores[-1]:.4f}")
19
20 # Verificar pesos de cada estimador
21 print(f"\nPeso del primer estimador (alpha_1): {adaboost.
    estimator_weights_[0]:.4f}")
```

19.4. Gradient Boosting

19.4.1. Marco General

Gradient Boosting (Friedman, 2001) es una generalización del Boosting que plantea el aprendizaje como un problema de optimización en el espacio funcional. En cada iteración, se añade un árbol que aproxima los **residuos negativos del gradiente** de la función de pérdida.

Para una función de pérdida diferenciable $L(y, F(x))$:

$$F_t(x) = F_{t-1}(x) + \eta \cdot h_t(x)$$

donde h_t se entrena para aproximar los pseudo-residuos:

$$r_{it} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{t-1}}$$

XGBoost y **LightGBM** son implementaciones altamente optimizadas que incorporan:

- Regularización L1 y L2 sobre los pesos de las hojas.
- Crecimiento de árboles por nivel (*level-wise*) o por hojas (*leaf-wise*).
- Soporte para valores faltantes de forma nativa.
- Paralelización eficiente.

Listing 7: Gradient Boosting clásico y XGBoost

```
1 from sklearn.ensemble import GradientBoostingClassifier
2 # pip install xgboost
3 from xgboost import XGBClassifier
4
5 # Gradient Boosting estandar
6 gb = GradientBoostingClassifier(
7     n_estimators=300,
8     learning_rate=0.05,      # Tasa de aprendizaje pequeña + más
9                               rboles = mejor
10    max_depth=3,             # rboles poco profundos son claves en
11                               boosting
12    subsample=0.8,           # Stochastic GB: muestra el 80% en cada
13                               rbol
14    max_features='sqrt',
15    random_state=42
16 )
17 gb.fit(X_train, y_train)
18
19 # XGBoost: implementación eficiente con regularización
20 xgb = XGBClassifier(
21     n_estimators=300,
22     learning_rate=0.05,
23     max_depth=4,
24     subsample=0.8,
25     colsample_bytree=0.8,    # Fracción de características por
26                               rbol
27     reg_alpha=0.1,          # Regularización L1
28     reg_lambda=1.0,         # Regularización L2
29     eval_metric='logloss',
30     random_state=42,
31     use_label_encoder=False
```

```
28 )
29 xgb.fit(X_train, y_train,
30         eval_set=[(X_test, y_test)],
31         early_stopping_rounds=20,
32         verbose=False)
33
34 print(f"GB Accuracy: {gb.score(X_test, y_test):.4f}")
35 print(f"XGB Accuracy: {xgb.score(X_test, y_test):.4f}")
```

19.5. Stacking (Apilamiento)

El Stacking utiliza las predicciones de modelos base (*nivel 0*) como características para entrenar un meta-modelo (*nivel 1*). Para evitar el sobreajuste, las predicciones del nivel 0 se generan mediante validación cruzada (*out-of-fold predictions*).

Listing 8: Stacking con múltiples modelos base

```
1 from sklearn.ensemble import StackingClassifier
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.svm import SVC
4 from sklearn.ensemble import RandomForestClassifier,
   GradientBoostingClassifier
5 from sklearn.neighbors import KNeighborsClassifier
6
7 # Definir modelos base (nivel 0)
8 base_models = [
9     ('rf', RandomForestClassifier(n_estimators=100, random_state=42))
10    ,
11     ('svm', SVC(kernel='rbf', probability=True, random_state=42)),
12     ('knn', KNeighborsClassifier(n_neighbors=7)),
13     ('gb', GradientBoostingClassifier(n_estimators=100, random_state
   =42))
14 ]
```

```
14
15 # Meta-modelo (nivel 1): aprende a combinar las predicciones base
16 meta_model = LogisticRegression(C=1.0, random_state=42)
17
18 # Stacking completo
19 stacking = StackingClassifier(
20     estimators=base_models,
21     final_estimator=meta_model,
22     cv=5,                # Validación cruzada para generar out-of-
                           fold predictions
23     stack_method='predict_proba', # Usar probabilidades, no solo
                           clases
24     n_jobs=-1
25 )
26 stacking.fit(X_train, y_train)
27 print(f"Stacking Accuracy: {stacking.score(X_test, y_test):.4f}")
```

19.6. Comparación de Métodos Ensemble

Cuadro 3: Comparación de técnicas de Ensemble Learning

Método	Estrategia	Reduce	Tiempo de entrenamiento	Riesgo de overfitting
Bagging	Paralelo promedio	+ Varianza	Bajo (paralelizable)	Bajo
Random Forest	Bagging features aleatorias	+ Varianza correlación	Moderado	Bajo
AdaBoost	Secuencial pesos	+ Bias	Moderado	Medio
Gradient Boosting	Secuencial gradiente	+ Bias	Alto (secuencial)	Medio-Alto
XGBoost	Gradient regularización	+ Bias + varianza	Moderado (optimizado)	Bajo-Medio
Stacking	Meta-aprendizaje	Ambos	Muy alto	Medio

20. Desarrollo de Sistemas de Machine Learning

20.1. El Pipeline Completo de ML

El desarrollo de sistemas de ML industriales no se limita al entrenamiento de un modelo. Según el marco presentado en Russell y Norvig (2022), el proceso completo abarca múltiples etapas interconectadas:

Datos brutos $\xrightarrow{\text{Recolección}}$ $\mathcal{D}$ $\xrightarrow{\text{Preprocesamiento}}$ $\tilde{\mathcal{D}}$ $\xrightarrow{\text{Modelado}}$ $\hat{f}$ $\xrightarrow{\text{Evaluación}}$ Métricas $\xrightarrow{\text{Despliegue}}$ Producción

20.2. Etapa 1: Definición del Problema y Recolección de Datos

Antes de recolectar datos, es crítico definir claramente:

- **Objetivo de negocio:** ¿Qué problema real se intenta resolver?
- **Función de pérdida:** ¿Qué métrica es relevante para el negocio?
- **Tipo de tarea:** clasificación, regresión, clustering, detección de anomalías.
- **Línea base:** ¿cuál es el rendimiento de la solución actual (humana o heurística)?

En el campo del reconocimiento de voz, por ejemplo, los sistemas modernos se entrenan con corpus de millones de horas de audio. La calidad y diversidad de los datos es frecuentemente más importante que la sofisticación del modelo (Banko y Brill, 2001).

20.3. Etapa 2: Análisis Exploratorio y Preprocesamiento

Listing 9: Pipeline de preprocesamiento con scikit-learn

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.pipeline import Pipeline
4 from sklearn.compose import ColumnTransformer

```



```
5 from sklearn.preprocessing import StandardScaler, OneHotEncoder,
    LabelEncoder
6 from sklearn.impute import SimpleImputer
7 from sklearn.feature_selection import SelectKBest, f_classif
8
9 # Identificar tipos de columnas
10 num_features = ['edad', 'salario', 'experiencia']
11 cat_features = ['departamento', 'ciudad', 'educacion']
12
13 # Pipeline para variables num ricas
14 num_pipeline = Pipeline([
15     ('imputer', SimpleImputer(strategy='median')),
16     ('scaler', StandardScaler())
17 ])
18
19 # Pipeline para variables categ ricas
20 cat_pipeline = Pipeline([
21     ('imputer', SimpleImputer(strategy='most_frequent')),
22     ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=
        False))
23 ])
24
25 # Combinar pipelines
26 preprocessor = ColumnTransformer([
27     ('num', num_pipeline, num_features),
28     ('cat', cat_pipeline, cat_features)
29 ])
30
31 # Pipeline completo con selecci n de features y modelo
32 full_pipeline = Pipeline([
33     ('preprocessor', preprocessor),
34     ('feature_selection', SelectKBest(f_classif, k=20)),
```

```

35     ('classifier', RandomForestClassifier(n_estimators=100,
36         random_state=42))
37
38 # Entrenar y evaluar
39 full_pipeline.fit(X_train, y_train)

```

20.4. Etapa 3: División de Datos y Validación

20.4.1. Estrategias de Validación

Hold-out: División simple en entrenamiento/validación/test (ej. 70/15/15).

k -Fold Cross-Validation: División en k folds para estimación robusta del error:

$$CV(k) = \frac{1}{k} \sum_{i=1}^k L(y^{(i)}, \hat{f}^{(-i)}(x^{(i)}))$$

Estratified k -Fold: Mantiene la proporción de clases en cada fold (recomendado para clases desbalanceadas).

Time-series Split: Para datos temporales, siempre validar en períodos futuros.

Listing 10: Validación cruzada con múltiples métricas

```

1 from sklearn.model_selection import (cross_validate, StratifiedKFold,
2     learning_curve, validation_curve
3     )
4
5 import numpy as np
6
7 # Validación cruzada con múltiples métricas
8 cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state
9     =42)
10
11 scoring = {
12     'accuracy': 'accuracy',

```

```
10     'precision': 'precision_weighted',
11     'recall': 'recall_weighted',
12     'f1': 'f1_weighted',
13     'roc_auc': 'roc_auc_ovr'
14 }
15
16 cv_results = cross_validate(
17     full_pipeline, X_train, y_train,
18     cv=cv_strategy,
19     scoring=scoring,
20     return_train_score=True,
21     n_jobs=-1
22 )
23
24 for metric, values in cv_results.items():
25     if metric.startswith('test_'):
26         print(f"{metric}: {values.mean():.4f}    {values.std():.4f}")
```

20.5. Etapa 4: Selección y Entrenamiento del Modelo

20.5.1. Estrategia de Selección

La selección del modelo adecuado depende de:

- **Tamaño del dataset:** k -NN y SVM funcionan bien con pocos datos; RF y XGBoost escalan bien.
- **Interpretabilidad requerida:** Árboles y Naive Bayes son más interpretables que Random Forest o XGBoost.
- **Tipo de datos:** SVM y RF manejan bien datos numéricos; Naive Bayes es natural para texto.

- **Recursos computacionales:** k -NN tiene alta latencia en predicción; SVM escala mal con $n > 100k$.

Listing 11: Comparación sistemática de modelos

```
1 from sklearn.model_selection import cross_val_score
2 from sklearn.metrics import classification_report, confusion_matrix
3
4 models = {
5     'KNN': KNeighborsClassifier(n_neighbors=7),
6     'SVM-RBF': SVC(kernel='rbf', C=10, probability=True),
7     'Decision Tree': DecisionTreeClassifier(max_depth=7, random_state
8         =42),
9     'Random Forest': RandomForestClassifier(n_estimators=200,
10        random_state=42),
11     'Gradient Boosting': GradientBoostingClassifier(n_estimators=200,
12        learning_rate
13            =0.05),
14     'AdaBoost': AdaBoostClassifier(n_estimators=200, learning_rate
15        =0.1)
16 }
17
18 results = {}
19 for name, model in models.items():
20     pipe = Pipeline([
21         ('preprocessor', preprocessor),
22         ('model', model)
23     ])
24     scores = cross_val_score(pipe, X_train, y_train, cv=5,
25        scoring='f1_weighted', n_jobs=-1)
26     results[name] = {'mean': scores.mean(), 'std': scores.std()}
27     print(f"{name:20s}: F1 = {scores.mean():.4f}    {scores.std():.4f}
28         ")
```

20.6. Etapa 5: Diagnóstico de Bias y Varianza

20.6.1. Curvas de Aprendizaje

El análisis de las curvas de aprendizaje es fundamental para diagnosticar problemas de *overfitting* (alta varianza) e *underfitting* (alto bias):

- **Underfitting (Alto Bias):** El error de entrenamiento y validación son ambos altos y similares. Solución: modelo más complejo, más características, menos regularización.
- **Overfitting (Alta Varianza):** El error de entrenamiento es bajo pero el de validación es alto. Solución: más datos, regularización, modelo más simple, dropout.
- **Buen ajuste:** Ambos errores son bajos y cercanos entre sí.

Listing 12: Análisis de curvas de aprendizaje

```
1 from sklearn.model_selection import learning_curve
2 import matplotlib.pyplot as plt
3
4 def plot_learning_curves(model, X, y, title):
5     train_sizes, train_scores, val_scores = learning_curve(
6         model, X, y,
7         train_sizes=np.linspace(0.1, 1.0, 10),
8         cv=5,
9         scoring='accuracy',
10        n_jobs=-1
11    )
12
13    train_mean = train_scores.mean(axis=1)
14    train_std = train_scores.std(axis=1)
15    val_mean = val_scores.mean(axis=1)
16    val_std = val_scores.std(axis=1)
17
18    plt.figure(figsize=(10, 6))
```

```

19 plt.plot(train_sizes, train_mean, 'o-', color='blue', label='
    Entrenamiento')
20 plt.fill_between(train_sizes, train_mean - train_std,
21                 train_mean + train_std, alpha=0.1, color='blue')
22 plt.plot(train_sizes, val_mean, 'o-', color='red', label='
    Validaci n')
23 plt.fill_between(train_sizes, val_mean - val_std,
24                 val_mean + val_std, alpha=0.1, color='red')
25
26 plt.xlabel('Tama o del conjunto de entrenamiento')
27 plt.ylabel('Accuracy')
28 plt.title(f'Curvas de Aprendizaje: {title}')
29 plt.legend()
30 plt.grid(True, alpha=0.3)
31 plt.savefig(f'curva_{title.lower().replace(" ", "_")}.png', dpi
    =150)
32
33 plot_learning_curves(rf_model, X, y, "Random Forest")

```

20.7. Etapa 6: Evaluación Final

20.7.1. Métricas de Evaluación

La selección de métricas debe alinearse con el objetivo de negocio:

Para clasificación:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{ROC-AUC} = P(\hat{y}_{pos} > \hat{y}_{neg})$$

Para regresión:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Listing 13: Evaluación completa del modelo final

```

1 from sklearn.metrics import (classification_report, confusion_matrix,
2                               roc_auc_score, roc_curve,
3                               precision_recall_curve,
4                               average_precision_score)
5
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8
9 # Predicciones finales SÓLO en el conjunto de test ( nunca antes!)
10 y_pred = best_model.predict(X_test)
11 y_proba = best_model.predict_proba(X_test)
12
13 # Reporte completo
14 print("=" * 60)
15 print("EVALUACIÓN FINAL DEL MODELO")
16 print("=" * 60)
17 print(classification_report(y_test, y_pred,
18                             target_names=class_names))
19
20 # ROC-AUC (multiclase)
21 roc_auc = roc_auc_score(y_test, y_proba, multi_class='ovr', average='
22                        weighted')
23 print(f"ROC-AUC (weighted): {roc_auc:.4f}")
24
25 # Matriz de confusi n

```

```
23 fig, ax = plt.subplots(figsize=(8, 6))
24 cm = confusion_matrix(y_test, y_pred)
25 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
26             xticklabels=class_names, yticklabels=class_names, ax=ax)
27 ax.set_xlabel('Predicci n')
28 ax.set_ylabel('Real')
29 ax.set_title('Matriz de Confusi n - Modelo Final')
30 plt.tight_layout()
31 plt.savefig('confusion_matrix.png', dpi=150)
```

20.8. Etapa 7: Despliegue y Monitoreo en Producción

El despliegue de un modelo de ML en producción requiere considerar aspectos más allá del rendimiento técnico.

20.8.1. Serialización del Modelo

Listing 14: Serialización y carga del modelo

```
1 import joblib
2 import json
3 from datetime import datetime
4
5 # Guardar el pipeline completo
6 model_info = {
7     'model_version': '1.0.0',
8     'training_date': datetime.now().isoformat(),
9     'test_accuracy': float(best_model.score(X_test, y_test)),
10    'features': feature_names,
11    'classes': class_names
12 }
13
14 # Guardar modelo y metadatos
```



```
15 joblib.dump(best_model, 'model_v1.0.0.pkl', compress=3)
16 with open('model_info.json', 'w') as f:
17     json.dump(model_info, f, indent=2)
18
19 # Carga en producci n
20 loaded_model = joblib.load('model_v1.0.0.pkl')
```

20.8.2. Monitoreo del Modelo

Una vez en producción, es fundamental monitorear:

- **Data drift**: cambios en la distribución de las variables de entrada.
- **Concept drift**: cambios en la relación entre variables de entrada y salida.
- **Model decay**: deterioro gradual del rendimiento con el tiempo.
- **Fairness**: equidad del modelo en diferentes subgrupos de la población.

20.9. Resumen del Pipeline Completo

Listing 15: Pipeline completo de ML production-ready

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.model_selection import train_test_split
3 from sklearn.metrics import accuracy_score, f1_score,
   classification_report
4 import pandas as pd
5 import joblib
6
7 # ---- 1. CARGA Y DIVISI N DE DATOS ----
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y, test_size=0.2, random_state=42, stratify=y
10 )
11
```

```
12 # ---- 2. PIPELINE COMPLETO ----
13 pipeline = Pipeline([
14     ('preprocessor', preprocessor),
15     ('model', best_model)
16 ])
17
18 # ---- 3. ENTRENAMIENTO ----
19 pipeline.fit(X_train, y_train)
20
21 # ---- 4. EVALUACION ----
22 y_pred = pipeline.predict(X_test)
23 print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
24 print(f"F1 Score: {f1_score(y_test, y_pred, average='weighted'):.4f}
25     ")
26
27 print(classification_report(y_test, y_pred))
28
29 # ---- 5. SERIALIZACION ----
30 joblib.dump(pipeline, 'production_model.pkl')
31 print("Modelo guardado exitosamente.")
```

21. Discusión Avanzada

21.1. El Balance Bias-Varianza en la Práctica

El dilema bias-varianza es uno de los conceptos más importantes del aprendizaje automático y aplica a todos los modelos estudiados:

$$E[(y - \hat{f}(x))^2] = \underbrace{[f(x) - E[\hat{f}(x)]]^2}_{\text{Bias}^2} + \underbrace{E[(\hat{f}(x) - E[\hat{f}(x)])^2]}_{\text{Varianza}} + \underbrace{\sigma^2}_{\text{Ruido irreducible}}$$

Las técnicas ensemble abordan este problema de distintas formas:

- **Bagging/Random Forest:** reducen la varianza promediando modelos de alta varianza (árboles profundos).
- **Boosting:** reducen el bias combinando secuencialmente modelos de alto bias (árboles superficiales).
- **Regularización:** Ridge, Lasso, dropout $\rightarrow$ reducen varianza introduciendo un pequeño bias controlado.

21.2. La Maldición de la Dimensionalidad

Un problema crítico para los modelos no paramétricos, especialmente k -NN, es que a medida que el número de dimensiones p crece, el volumen del espacio aumenta exponencialmente, haciendo que los datos se vuelvan **escasos**. El volumen de una hiperesfera de radio r en p dimensiones es:

$$V_p(r) = \frac{\pi^{p/2}}{\Gamma(p/2 + 1)} r^p$$

Para mitigar este efecto se utilizan técnicas de reducción de dimensionalidad como PCA, t-SNE, UMAP o selección de características.

21.3. Importancia en la Industria

Tal como documenta el Estudio de los Cien Años sobre IA de Stanford (AI100), los sistemas de ML han demostrado aplicaciones revolucionarias en numerosos sectores:

- **Medicina:** Algoritmos de ML que igualan o superan a médicos expertos en diagnóstico por imagen (detección de cáncer, enfermedades oftalmológicas, diagnóstico de Alzheimer). El sistema LYNA alcanza una precisión del 99.6 % en diagnóstico de cáncer de mama metastásico.
- **Finanzas:** Detección de fraude en tiempo real, scoring crediticio, trading algorítmico, gestión de riesgos.
- **Procesamiento de lenguaje natural:** Traducción automática en más de 100 idiomas para cientos de millones de usuarios diarios.
- **Visión artificial:** Reconocimiento de objetos con tasas de error que bajaron del 28 % en 2010 al 2 % en 2017, superando el rendimiento humano.
- **Juegos:** AlphaGo y AlphaZero demostrando aprendizaje sin supervisión humana para dominar el Go, el ajedrez y el shogi.

22. Conclusiones

Este trabajo ha presentado de manera integral tres pilares del aprendizaje automático avanzado:

1. Modelos No Paramétricos: Los algoritmos k -NN, SVM, árboles de decisión y Naive Bayes ofrecen herramientas flexibles que no imponen restricciones sobre la forma funcional de los datos. Cada uno presenta fortalezas específicas: SVM destaca en espacios de alta dimensión gracias al truco del kernel; los árboles de decisión ofrecen alta interpretabilidad; k -NN es trivialmente simple pero computacionalmente costoso. La clave está en elegir el modelo adecuado según el problema, el tamaño del dataset y los requisitos de interpretabilidad.

2. Ensemble Learning: La combinación de múltiples modelos base supera sistemáticamente a cualquier modelo individual. Random Forest es el ensemble más robusto y de uso general, aprovechando la aleatoriedad para reducir la varianza. Los métodos de Boosting (AdaBoost, Gradient Boosting, XGBoost) reducen el bias iterativamente y son frecuentemente los mejores en competencias de ML supervisado. El Stacking representa la técnica más sofisticada, capaz de combinar modelos heterogéneos mediante meta-aprendizaje.

3. Desarrollo de Sistemas de ML: El pipeline completo — desde la definición del problema hasta el monitoreo en producción — es tan importante como la elección del algoritmo. El preprocesamiento riguroso, la validación cruzada, el diagnóstico bias-varianza y la serialización correcta son componentes indispensables de cualquier sistema ML de calidad industrial.

Estos conocimientos son relevantes no solo en el contexto académico, sino en las aplicaciones prácticas descritas por Russell y Norvig (2022), donde los sistemas de IA actuales ya demuestran rendimiento a nivel humano o superior en diagnóstico médico, reconocimiento de voz, traducción y juegos estratégicos, impulsando transformaciones en medicina, finanzas, transporte y prácticamente todos los sectores productivos.

El futuro del aprendizaje automático apunta hacia modelos más robustos, interpretables y equitativos, con un énfasis creciente en la colaboración humano-máquina y en garantizar la privacidad, transparencia y ausencia de sesgos en los sistemas desplegados.

Referencias

- Banko, M., & Brill, E. (2001). Scaling to very very large corpora for natural language disambiguation. *Proceedings of the 39th Annual Meeting of the Association for Computational Linguistics*.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
- Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.
- Russell, S., & Norvig, P. (2022). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Silver, D., et al. (2017). Mastering the game of Go without human knowledge. *Nature*, 550(7676), 354–359.
- Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2), 241–259.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.